

GitHub Innovation Portal

A Real-Time Open-Source Trend Analytics Platform

Jann Erhardt

erharjan@students.zhaw.ch

Zurich University of Applied Sciences
Winterthur, Zurich, Switzerland

Gian Gamper

gamepegia@students.zhaw.ch

Zurich University of Applied Sciences
Winterthur, Zurich, Switzerland

Jonas Bratschi

bratsjon@students.zhaw.ch

Zurich University of Applied Sciences
Winterthur, Zurich, Switzerland

Abstract

Open-source activity on GitHub generates tens of millions of events per day, making it difficult for developers, researchers, and technology scouts to identify promising early-stage projects before they gain mainstream attention. This paper presents the *GitHub Innovation Portal*, an end-to-end streaming analytics platform that ingests the public GitHub event stream in near-real time, enriches each event with geographic and user-profile metadata, stores it in a time-series optimised database, and exposes actionable insights through two complementary visualisation layers.

The system is built on Apache Kafka (KRaft) for durable event buffering, TimescaleDB for compressed hypertable storage with pre-computed continuous aggregates, FastAPI for a low-latency REST and Server-Sent Events interface, and a Next.js 16 dashboard for live trend visualisation. The entire stack is containerised and deployable with a single `docker compose` command. **[TODO: Refine after report completion]**

CCS Concepts

• **Information systems** → **Data streams**; *Database management system engines*; • **Software and its engineering** → Software architectures.

Keywords

stream processing, GitHub event API, Apache Kafka, TimescaleDB, real-time analytics, open-source trend detection, geo-enrichment, Server-Sent Events, Next.js dashboard

ACM Reference Format:

Jann Erhardt, Gian Gamper, and Jonas Bratschi. 2025. GitHub Innovation Portal: A Real-Time Open-Source Trend Analytics Platform. In *Proceedings of Big Data Projects (PM4 FS25)*. ACM, New York, NY, USA, 11 pages.

1 Introduction

The open-source ecosystem hosted on GitHub grows continuously, with millions of push events, pull requests, issue comments, and repository forks published every day [2]. For developers, researchers, and technology scouts this volume presents a paradox: the very richness of the signal makes it practically impossible to manually track emerging projects before they appear in mainstream media.

Existing discovery mechanisms rely on lagging indicators — stars accumulated over weeks, blog posts, or curated lists — rather than on the raw momentum visible in the live event stream. A repository that doubles its commit frequency or attracts contributors from three continents in a single day represents a signal of genuine innovation; today that signal is largely wasted.

To address this, the system implements a *Hidden Gem discovery engine* that assigns a statistical significance score to each repository based on observed star and fork activity relative to its expected baseline rate. The engine applies a Poisson significance test across three time windows (24 h, 168 h, 730 h) and exposes ranked results through a dedicated web interface and REST API. Repositories, their owning users, and organisations are scored in parallel; snapshots are persisted at each interval to enable temporal trend analysis and cohort evaluation.

This project addresses the gap with the *GitHub Innovation Portal*: an end-to-end streaming analytics platform that continuously ingests the GitHub public events API, enriches each event with geographic and user-profile context, persists the data in a time-series optimised store, and surfaces trends through an interactive dashboard and operational Grafana dashboards.

The remainder of this paper is structured as follows. Section 2 surveys related approaches. Section 3 describes the system architecture. Section 4 details the implementation of each component. Section 5 presents evaluation results. Section 6 justifies the technology choices. Section 7 discusses challenges encountered. Section 8 concludes with lessons learned and future work.

2 Related Work

Detecting emerging open-source projects before they gain widespread attention requires both historical context and the ability to process high-velocity event streams. Several prior systems address these challenges from complementary angles.

GitHub Archive and GHTorrent. GH Archive [4] pioneered the idea of recording GitHub’s public timeline by crawling the `GET /events` endpoint every hour and publishing the raw JSON payloads as gzip-compressed files on Google Cloud Storage. This initiative enabled retrospective analyses of GitHub’s growth and contributor behaviour over years of data [7]. GHTorrent (GitHub Torrent) extended this concept by not only mirroring events but also performing a one-time full dump of repository and user metadata from the GitHub REST API and maintaining a live-updated MongoDB instance with the enriched dataset [3]. While GH Archive and GHTorrent provide valuable historical depth, neither system offers near-real-time ingestion or a pre-computed analytics layer.

Streaming architectures. The Lambda architecture [8] proposes batch and speed layers running in parallel, with a serving layer merging their results. The Kappa architecture [6] simplifies this by treating the batch layer as a special case of stream processing, using a single unified pipeline. Apache Kafka Streams and Apache Flink are representative frameworks that enable stateful stream processing with exactly-once semantics. These patterns are

well-established in the literature on real-time analytics but have only recently been applied to open-source ecosystem monitoring.

Time-series databases. TimescaleDB, InfluxDB, and ClickHouse are frequently compared for time-series workloads. Prior work on time-series database selection highlights TimescaleDB's advantage in hybrid workloads where SQL expressiveness and time-based compression are both required [10]. This motivates the storage choice in our pipeline.

Open-source trend detection. GitHub's own Trending page surfaces repositories with the highest star velocity over a configurable time window, but it is limited to stars and does not expose the underlying event stream. Third-party services such as *OSS Insight* and *GitHub Star History* offer curated visualisations of repository popularity but are read-only analytical tools without ingest or enrichment capabilities. To our knowledge, no existing system combines continuous GitHub event ingestion, per-event geographic enrichment, and sub-minute latency visualisation in a single deployable pipeline.

3 System Architecture

The architecture of the project consists of a real-time data streaming pipeline designed to ingest, enrich, and visualize GitHub events. It is built to efficiently manage GitHub API rate limits and ensure high scalability by decoupling individual components using a message broker.

3.1 Data Ingestion (Producer)

The **Producer** (`producer/producer.py`) serves as the entry point of the pipeline:

- **Polling:** It polls the public GitHub Events API every 10 seconds.
- **Deduplication:** To prevent redundant data, the producer filters events based on their unique GitHub Event ID.
- **Kafka Integration:** Cleaned raw JSON data is published to the Kafka topic `github.events.raw`.
- **Rate Limit Management:** A standard GitHub personal access token has a rate limit of 5 000 requests per hour, which is sufficient for continuous polling without hitting the unauthenticated cap of 60 requests per hour.

3.2 Message Broker (Kafka)

Kafka acts as the central message bus, operating in KRaft mode without the need for Zookeeper:

- **Decoupling:** It separates the high-frequency data ingestion (Producer) from the more complex data enrichment steps (Consumer).
- **Durability:** Kafka serves as a reliable buffer; if a consumer fails, data is retained for 48 hours, allowing the system to resume where it left off.
- **Scalability:** The main topics are divided into three partitions, enabling parallel processing by multiple consumer instances.
- **Topics:** Four topics carry distinct payload types: `github.events.raw` (raw event JSON), `github.events.status` (per-request enrichment status), `github.ratelimit` (rate-limit snapshots), and `geocoder.requests` (geocoder request logs, single partition).

3.3 Processing and Enrichment (Consumer & Geocoder)

In this stage, raw events are transformed into high-value data through a highly optimized enrichment process. The system is designed to maximize throughput while respecting strict external API constraints.

- **Parallel Consumer Instances:** The system is designed for horizontal scalability. Multiple consumer instances (e.g., three instances in multi-consumer mode) can run in parallel. Using Kafka's partition assignment, each instance is "pinned" to specific partitions, ensuring that event processing is distributed evenly and without overlap.
- **Token Pool Management:** To overcome GitHub's rate limits, the system implements a **TokenPool** mechanism. It manages multiple API keys for both user and repository metadata. The pool uses a round-robin strategy and automatically skips tokens that have been flagged as rate-limited, resuming their use only after their specific reset time has passed.
- **GraphQL Batch Processing:** The primary enrichment method uses the **GitHub GraphQL API**. Instead of requesting data for every event individually, the **Enricher** accumulates up to 20 items (users or repositories) and fetches them in a single batched request. This significantly reduces the number of required HTTP calls and maximizes the utility of each API token.
- **REST Fallback Mechanism:** While GraphQL is the preferred method for efficiency, the system includes a **REST API fallback**. If a GraphQL batch query returns null for a specific entity, the system automatically issues an individual REST call for that entity.
- **Geocoding (Geocoder & Photon):** Geographic data is processed by a dedicated geocoder service. It extracts location strings from enriched user profiles and queries the local **Photon** instance. This allows the system to resolve coordinates and country codes in the background without blocking the main event-processing pipeline.

3.4 Data Storage (TimescaleDB)

The project uses **TimescaleDB** (a PostgreSQL extension) as its primary storage solution:

- **Hypertables:** Event data is automatically partitioned into time-based chunks, which significantly improves query performance for large datasets.
- **Continuous Aggregates:** The database automatically precalculates rollups (e.g., 5-minute statistics) in the background, allowing complex dashboard queries to load in milliseconds.

The relational schema is organised into three groups — the core star schema (Figures 3), the Hidden Gem snapshot tables (Figure 4), and the operational log tables (Figure 5) — documented in Appendix C.

3.5 Visualization

The system provides two complementary visualisation layers. **Grafana** serves a dual purpose: its analytics dashboards (the *Hidden Gem* series) provide deep-dive insights into the enriched GitHub data,

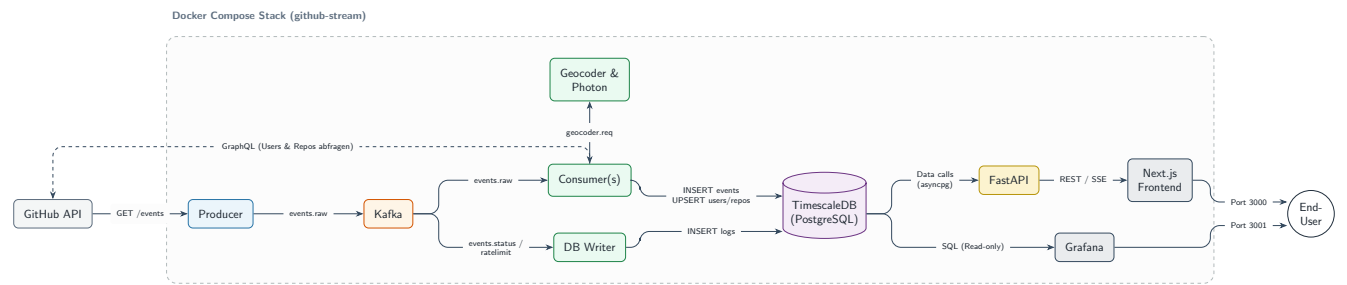


Figure 1: End-to-end architecture of the GitHub Innovation Portal. Events flow from the GitHub API through Kafka (KRaft) into TimescaleDB, where they are served by FastAPI to the Next.js dashboard and Grafana. Dashed arrows indicate authentication and auxiliary data flows. See Appendix A for the full annotated diagram.

while its monitoring dashboards expose pipeline health, geocoding throughput, and API request logs to operators. The **Next.js dashboard** (Section 4) provides the interactive, real-time analytics interface for end users, including the Hidden Gem discovery and ranking features. The Grafana dashboards are categorized into two main areas:

- GitHub Data Insights (Hidden Gem Series):** These dashboards provide deep-dive analytics into the enriched GitHub data:
 - Hidden Gem - Overview:* A high-level summary of all captured events and geographic distributions.
 - Hidden Gem - Organizations:* Metrics focused on organizational activity and trends.
 - Hidden Gem - Repositories:* Analysis of repository growth, popularity, and activity.
 - Hidden Gem - Users:* Insights into user contributions and profiles.
- System Monitoring and Status:** Dedicated dashboards track the operational health and performance of the pipeline:
 - System Status Overview:* Monitoring of database performance and overall system health.
 - Geocoder Request Logs:* Real-time tracking of the geocoding service’s efficiency and request volume.
 - General Request Logs:* Logs and metrics related to internal data flow and API interactions.

3.6 Geocoding Engine (Photon)

To implement high-performance geographic enrichment of GitHub events without relying on external rate-limited services, **Photon** was integrated into the infrastructure.

- Local Instance:** Photon is an open-source geocoding service based on OpenStreetMap data. By operating it as a local Docker container, latency is minimized and the strict usage constraints of public APIs (such as Nominatim) are bypassed.
- Data Basis and Regions:** The service is initialized with specific datasets—for instance, covering regions like Africa—which are persisted in the photon_data volume to ensure stability and speed.
- API Interface:** Photon provides a REST API that is queried by the geocoder service. It processes unstructured location

strings from GitHub profiles and returns structured JSON data, including latitude, longitude, and ISO country codes.

- Fault Tolerance:** The local instance’s high performance allows for complex fuzzy search queries. This ensures that even if a location cannot be resolved immediately, the overall data flow of the pipeline is not delayed.

Table 1: Expanded System Component Overview (Docker Services)

Service	Function
producer	Polls the GitHub API and writes raw event data to Kafka
kafka	Central message broker used for service decoupling and
kafka-ui	Web UI (port 8080) for Kafka topic inspection and consu
kafka-init	One-shot service that creates Kafka topics with the correc first start.
consumer-{0,1,2}	Three parallel instances that enrich events with user and each is pinned to one Kafka partition.
geocoder	Logic layer that claims unlabelled user locations from the via Photon.
photon	Local geocoding engine (OpenStreetMap/Photon) that res to coordinates.
db-writer	Consumes status and rate limit topics to persist operatio
timescaledb	Time-series optimized SQL database used for data storag
grafana	Operational monitoring dashboards for pipeline health a

3.7 Trending Repository Discovery

The trending repository discovery module operates as an analytical layer on top of the enriched event data stored in TimescaleDB. It consumes the events and repos tables produced by the consumer and geocoding services and exposes its results through the FastAPI layer and the Next.js frontend.

The module comprises three components. First, a set of PostgreSQL aggregate functions (hidden_gem_repo_scores, hidden_gem_user_score, hidden_gem_org_scores) compute per-entity significance scores on demand from the raw event tables. Second, a snapshot scheduler materialises these scores into persistent snapshot tables at configurable intervals (24 h, 168 h, 730 h). Third, a dedicated FastAPI

sub-router exposes filtered rankings, entity detail views, and cohort evaluation endpoints.

This design keeps scoring logic entirely within the database tier, where TimescaleDB can leverage time-based indexing and column compression. No additional microservice is required; the scheduler runs as a background thread within the FastAPI container.

4 Implementation

4.1 Data Ingestion

The producer (`producer/producer.py`) fetches up to three pages of the `GET /events` endpoint per poll cycle, yielding approximately 90 events every 10 seconds. An in-process set of recently seen event IDs prevents duplicate Kafka messages when polling overlaps the GitHub API cache window. Providing a GitHub personal access token raises the unauthenticated rate limit from 60 to 5 000 requests per hour [2].

4.2 Event Enrichment

The consumer (`consumer/consumer.py`) feeds two independent batch queues inside an `Enricher` instance, each backed by its own GitHub token pool:

- (1) **User/organisation-profile enrichment.** Actor usernames are enqueued in a `user/org` batch queue (up to 20 items per GraphQL request). Before enqueueing, the `Enricher` checks the database for an existing row with `fetch_at IS NOT NULL`; already-enriched actors are skipped. A REST fallback handles GraphQL partial failures and 404 responses.
- (2) **Repository enrichment.** Repository IDs follow the same pattern through a separate `repo` batch queue with its own token pool. Resolved metadata (language, topics, star/fork counts) is upserted into the `repos` table. Geographic enrichment of user location strings is handled by the dedicated geocoder service (Section 3.6).

Enriched records are written to five normalised tables: `users`, `repos`, `organizations`, `organization_members`, and `events`.

4.3 Storage Layer

The `events` table is a TimescaleDB hypertable partitioned into one-day chunks. Chunk-level columnar compression is applied after seven days (order by time DESC, segment by event_type), achieving compression ratios of 10–20× over the raw row-store [10]. A 90-day data retention policy bounds disk usage.

Two continuous aggregates pre-compute rollups at ingest time rather than query time (Table 2), keeping dashboard query latency below a few milliseconds even at scale.

Table 2: TimescaleDB continuous aggregates

View	Bucket	Content
<code>event_stats_5m</code>	5 min	Event counts by event_type
<code>country_ids_5m</code>	5 min	Event counts by ISO country code

4.4 API Layer

The FastAPI service exposes the endpoints listed in Table 3. The `/stream/events` endpoint implements the Server-Sent Events protocol [5]: it queries the `events` table (joined with `repos`) every 3 seconds and writes JSON-encoded event objects to the response stream. An initial `: heartbeat` comment is written immediately to register the connection, followed by `: keep-alive` comments between data frames to prevent browser and proxy timeouts.

Table 3: FastAPI endpoint reference

Endpoint	Description
<code>GET /health</code>	Liveness check
<code>GET /api/kpis</code>	KPI metrics (commits, PRs, contributors, avg review time)
<code>GET /api/commits-over-time</code>	Daily commit buckets (configurable window)
<code>GET /api/top-repos</code>	Most active repositories (24 h window)
<code>GET /api/recent-events</code>	Latest ingested events
<code>GET /stream/events</code>	SSE live stream (3 s poll)
<code>GET /api/overview/heatmap</code>	Event counts by country for choropleth map
<code>GET /api/overview/globe-heatmap</code>	Globe-projection heatmap data
<code>GET /api/activity/leaderboard</code>	Top actors by event count (configurable window)

4.5 Visualisation

The Next.js [11] dashboard (port 3000) uses React 19 server components to fetch KPI and chart data at request time via the `API_URL` environment variable (internal Docker DNS). Client components subscribe to the SSE stream using a typed `useSSE` React hook that reconnects automatically after a 3-second back-off. The UI is built with Tremor v3 (Recharts under the hood) and Tailwind CSS 4.

In addition to the general analytics dashboard, the frontend provides a dedicated *Hidden Gems* section comprising four pages. The main ranking page presents a paginated leaderboard switchable between three entity scopes (repositories, users, organisations) and three time windows (24 h, 168 h, 730 h). For the repository scope, users can apply attribute filters on programming language, software licence, and topic tags. A global search bar with scope selection enables cross-entity lookup across all snapshot data.

Individual detail pages are available for repositories, users, and organisations. Each page displays the entity’s current significance score alongside a time-series chart of score history derived from successive snapshots. The user detail page additionally lists the entity’s scored repositories with their individual scores. The organisation detail page presents two independent score aggregates: one for repositories directly owned by the organisation and one for repositories owned by its registered members. An *Evaluation*

Reports view embedded in the main ranking page exposes cohort analysis results: for a selected snapshot, each previously detected repository is classified as *sustained* (significance score ≥ 1.5 in the subsequent same-interval snapshot) or *dropped* (score fell below threshold), with score trajectories shown.

4.6 Hidden Gem Scoring Engine

4.6.1 Momentum Metric. We define *momentum* for a repository r in a time window $[t - \Delta, t]$ as the weighted count of starring and forking events $k = \lfloor \alpha \cdot k_{\star} + \beta \cdot k_{\text{fork}} \rfloor$ observed in that window, where α and β are configurable coefficients (default: $\alpha = \beta = 1.0$). Raw momentum counts are not directly comparable across repositories: a repository with 50,000 cumulative stars receiving 100 new stars in 24 h reflects qualitatively different behaviour from a two-month-old repository with 300 cumulative stars receiving the same count. The scoring therefore normalises observed growth against an expected rate derived from the full event history of each repository.

4.6.2 Statistical Scoring Model.

Expected rate. Let λ denote the expected weighted event count in the window under a stationary Poisson null model:

$$\lambda = \frac{\alpha \cdot S_{\text{base}} + \beta \cdot F_{\text{base}}}{\max(A_r, 1)} \times \min(\Delta, A_r)$$

where S_{base} and F_{base} are the cumulative star and fork counts accrued before the window, A_r is the repository age in hours at the start of the window, and Δ is the window duration in hours. For repositories younger than the window ($A_r \leq \Delta$), the factor $\min(\Delta, A_r) = A_r$ cancels the denominator, reducing λ to the full historical baseline. This prevents repositories with limited history from receiving inflated scores due to a short observation period.

Significance score. Let $X \sim \text{Poisson}(\lambda)$ and k be the observed weighted event count. The Poisson survival function is $\text{SF}(k) = P(X \geq k; \lambda)$. The significance score is defined as:

$$\text{sig_score} = \min\left(\frac{-\ln \text{SF}(k)}{34.5} \times 10, 10\right)$$

The score equals zero when k is consistent with the expected rate and grows as observed growth becomes increasingly improbable under the null model, with a ceiling of 10. The constant 34.5 normalises the dynamic range to this maximum. A score of 3.0 corresponds to $\text{SF} \leq e^{-10.35} \approx 3.2 \times 10^{-5}$, the primary classification boundary used by the frontend to designate a repository as a genuine hidden gem. The survival function is computed in log-space via an iterative accumulation scheme (`F_poisson_sf()` in `004_hidden_gem_functions.sql`) to avoid numerical underflow for large k or small λ .

Noise guards. Repositories must satisfy minimum activity thresholds within the window (default: $k_{\star} \geq 5$, $k_{\text{fork}} \geq 1$) before a score is assigned. Repositories that do not meet these guards receive a NULL score and are excluded from rankings. The thresholds are configurable via environment variables.

4.6.3 User and Organisation Aggregation. Significance scores aggregate from repositories to users and organisations. For users, the aggregate function sums all repository scores within the window

(`total_score`), records the highest individual score (`best_repo_score`), and counts repositories that exceed the `sig_score` ≥ 3.0 boundary (`hidden_gem_count`). For organisations the aggregation produces two independent sub-totals: one for repositories directly owned by the organisation, and one for repositories owned by registered members of the organisation (resolved via the `organization_members` table). This split allows an organisation to assess its own output and the collective contribution of its member community independently.

4.6.4 Snapshot Scheduler. Because the aggregate queries scan the full events table, results are materialised at fixed intervals by the snapshot scheduler (`api/scheduler/snapshot_scheduler.py`), implemented with APScheduler. Three intervals are configured: 24 h, 168 h, and 730 h. On each run the scheduler inserts a metadata record into `hidden_gem_snapshot_runs`, fetches fresh scores from all three aggregate functions, bulk-inserts results into the corresponding snapshot tables (`hidden_gem_snapshot_repos`, `hidden_gem_snapshot_users`, `hidden_gem_snapshot_orgs`), and updates the run record with per-entity counts. Transient database errors trigger exponential back-off with a timeout scaled linearly by attempt number. Stored snapshots serve two purposes: they supply the score history data consumed by the frontend detail pages, and they enable the cohort evaluation described in Section 5.5.

4.6.5 API Layer. Eleven REST endpoints under `/api/hidden-gems/` expose the discovery engine. Live rankings (`/live`) carry a 30-second cache TTL to amortise aggregate query cost; filter metadata endpoints use a 5-minute TTL. A manual trigger endpoint (`POST /snapshots/trigger`) is restricted to administrative clients and allows on-demand snapshot creation outside the regular schedule.

[TODO: Insert table of Hidden Gem API endpoints (method, path, query parameters, description, cache TTL) here. Mirror the format of Table 3. Add a `\label{tab:hg-api}` so it can be referenced.]

5 Evaluation

TODO: Fill in this section after a representative system run (24–48 hours recommended). Suggested structure below.

5.1 Experimental Setup

5.1.1 Hardware and Operating System. The entire pipeline was hosted on a dedicated virtualized server. The system leverages high-performance AMD EPYC processors and sufficient memory to handle the concurrent execution of multiple Docker containers, including the Kafka broker and the TimescaleDB instance. The detailed system specifications are summarized in Table 4.

5.1.2 Software Environment and Containerization. The architecture is fully containerized using **Docker** and managed via **Docker Compose**. This approach ensures service isolation and simplified dependency management. The primary software components and their respective versions are:

- **Message Broker:** Apache Kafka v3.9.0, running in KRaft mode for metadata management without Zookeeper.

Table 4: System Specifications

Component	Specification
Hostname	github-insights
Operating System	Ubuntu 24.04.4 LTS (Noble)
Kernel	Linux 6.17.0-23-generic (x86_64)
CPU	AMD EPYC-Milan (16 vCPUs, 16 Cores)
Hypervisor	KVM (Full Virtualization)
Memory	31 GiB Total
Storage	348 GB (NVMe-based Virtual Disk)

- **Database:** TimescaleDB (based on PostgreSQL 16), utilizing hypertables and continuous aggregates for time-series optimization.
- **Geocoding Engine:** Photon (OpenStreetMap-based) running as a local REST service.
- **Runtime Environment:** Python 3.11+ for the Producer, Consumer, and Geocoder microservices.

5.1.3 *Pipeline Configuration.* For the experimental runs, the system was configured in **multi-consumer mode** to test horizontal scalability.

- **Parallelism:** Three independent consumer instances were deployed, each pinned to a specific Kafka partition to prevent processing overlaps.
- **Resource Allocation:** Given the 16 vCPUs available, the load was distributed across the Kafka broker, the TimescaleDB background workers, and the enrichment consumers to maintain a load average below 1.0 during peak ingestion.
- **Network:** A bridge-driver Docker network with a custom MTU of 1442 was used to optimize internal container communication.

5.2 Throughput and Latency

Table 5 summarises the key performance metrics measured during the evaluation period.

5.3 Data Coverage and Enrichment Quality

Table 6 shows the breadth of the data collected.

5.4 Storage Efficiency

The events hypertable currently spans 94 one-day chunks. Of these, 83 chunks (all data older than 7 days) have been compressed; 11 recent chunks remain in uncompressed row-store format. Table 7 summarises the measured storage figures.

The 2.9× overall ratio reflects the mixed column composition: structured fields (time, event_type, actor_username, repo_id) compress to a small fraction of their original row-store size (up to 36× on table data alone), while the payload JSONB column stored in PostgreSQL TOAST is already partially compressed internally and benefits less from columnar reordering, moderating the aggregate figure.

Table 5: Pipeline throughput and latency metrics

Metric	Value	Notes
Events ingested (total)	21,748,215	Over the full DB time range
Average ingestion rate	10.7 ev/min	Computed over full DB range
Peak ingestion rate	660 ev/min	
Median API resp. time (GraphQL)	0.294 s	1,754,020 requests; p95 = 0.668 s
Median API resp. time (REST)	0.303 s	5,592,045 requests; p95 = 0.551 s
Kafka consumer lag (steady)	0 msgs	All partitions at log-end offset

Table 6: Data coverage over evaluation period

Dimension	Value
Unique repositories	5,367,217
Unique contributors	3,963,190
Countries represented	217
Geocoding success rate	98.0%
Event types observed	CommitCommentEvent, CreateEvent, DeleteEvent, DiscussionEvent, ForkEvent, GollumEvent, IssueCommentEvent, IssuesEvent, MemberEvent, PublicEvent, PullRequestEvent, PullRequestReviewCommentEvent, PullRequestReviewEvent, PushEvent, ReleaseEvent, WatchEvent (16 types)
Most active programming lang.	HTML (824,115 repos)

5.5 Hidden Gem Detection Quality

5.5.1 *Cohort Analysis Methodology.* To assess detection quality without external ground truth, we apply a cohort evaluation approach: a repository detected in snapshot run *i* with sig_score ≥ 1.5 is classified as *sustained* if in the immediately following snapshot of the same interval it again achieves sig_score ≥ 1.5; otherwise it is classified as *dropped*. This methodology is implemented in the /api/hidden-gems/snapshots/{id}/cohort endpoint and is directly accessible via the *Evaluation Reports* view of the frontend. Sustained detections are interpreted as genuine momentum signals; dropped detections indicate transient activity spikes.

Table 7: events hypertable compression statistics

Metric	Value
Total chunks	94
Compressed chunks	83
Uncompressed chunks (recent)	11
Before compression (83 chunks)	12.75 GB
After compression (83 chunks)	4.43 GB
Overall compression ratio	2.9×

Table 8: Cohort evaluation results by time window. A repository is *sustained* if it re-achieves `sig_score` ≥ 1.5 in the immediately following snapshot of the same interval; otherwise it is *dropped*. Precision = sustained / flagged.

Window	Snaps.	Flagged	Sust.	Drop.	Prec.
24 h	17	64	38	26	59.4%
168 h	10	465	406	59	87.3%
730 h	10	993	888	105	89.4%

5.5.2 Representative Detections. Table 9 lists the five repositories with the highest significance scores recorded across all snapshot runs. All five reached the score ceiling of 10, indicating a Poisson survival probability of $SF \leq e^{-34.5} \approx 10^{-15}$ — far below the primary classification threshold. The score ceiling is reached by a notable fraction of detected repositories, meaning rank within the top tier must be broken by secondary criteria such as absolute star counts or window length.

5.5.3 Score Distribution Across Snapshots. [TODO: Report the distribution of `sig_score` values across all stored snapshot runs, broken down by time window (24 h, 168 h, 730 h). Report: number of scored repositories, mean, median, 90th-percentile `sig_score`, fraction with `sig_score` ≥ 3.0 , and fraction with `sig_score` ≥ 1.5 . Values can be obtained by querying the `hidden_gem_snapshot_repos` table directly.]

6 Technology Decision Rationale

Apache Kafka. Kafka [1] decouples the rate-limited GitHub poller from the slower geo-enrichment step. Its consumer group model allows the enrichment workload to be parallelised by running additional consumer containers without modifying the producer. The durable log acts as a replay buffer: if the consumer crashes mid-processing, it resumes from its last committed offset without data loss.

TimescaleDB over plain PostgreSQL. Plain PostgreSQL lacks automatic time-based chunk pruning and columnar compression; queries over large event tables become progressively slower as data accumulates. TimescaleDB [10] adds hypertables (transparent partitioning by time), 10–20× compression, and continuous aggregates that materialise rollups at ingest time. It retains full SQL compatibility, which is essential for the JOIN-heavy enrichment queries and JSONB payload fallback.

FastAPI over Flask/Django. FastAPI’s [9] async-first design and native `StreamingResponse` make SSE implementation straightforward. The automatic OpenAPI documentation at `/docs` was valuable during development and testing.

Next.js 16 (App Router). Server components eliminate the waterfall of client-side data fetches on initial page load. The App Router’s layout system provides a persistent sidebar without duplicating markup across pages. Tremor’s pre-built chart components (AreaChart, BarChart, Table) accelerated dashboard development significantly.

7 Challenges and Solutions

GitHub API rate limiting. Without authentication, the REST API caps unauthenticated requests at 60 per hour — insufficient for continuous polling. We mitigated this with an optional personal access token (raising the cap to 5 000/h) and client-side deduplication that avoids re-publishing events already seen within the polling window.

Geocoding deduplication across consumers. GitHub user location strings are free-form text, leading to many duplicates across events. With three concurrent consumer instances all triggering geocoding, naive processing would issue redundant Photon requests for the same location string simultaneously. We resolved this with an optimistic-locking claim: before forwarding a location to Photon, the geocoder sets `geo_claimed_at = NOW()` on the user row in a single `UPDATE . . . WHERE geo_claimed_at IS NULL` statement. Concurrent instances racing on the same username lose the claim and skip the request, ensuring each location is resolved exactly once without requiring a distributed lock.

Service startup ordering. TimescaleDB and Kafka require 10–30 seconds after container start before accepting connections. Docker Compose health checks gate dependent services. Additionally, the FastAPI service implements an internal retry loop that polls the database connection every three seconds and logs progress until the pool is established.

React 19 / Tremor peer dependency conflict. Tremor v3 declares a React 18 peer dependency while the project uses React 19. Installing with `-legacy-peer-deps` bypasses the version check; no runtime incompatibilities were observed during development.

[TODO: Additional challenges.] [TODO: Add any further challenges encountered during the project, e.g., Kafka KRaft configuration, TimescaleDB compression policy tuning, SSE browser compatibility, etc.]

8 Conclusion and Future Work

[TODO: Write 1–2 paragraphs. Summarise what was achieved relative to the three project objectives: (O1) defining momentum metrics, (O2) building a live dashboard, and (O3) granular filtering. Mention the most interesting finding from the data.]

The GitHub Innovation Portal demonstrates that a small team can build a production-grade, end-to-end streaming analytics platform using entirely open-source components within a single academic semester. The combination of Kafka for durable buffering,

Table 9: Top-5 repositories by peak significance score over the evaluation period ($\Delta\star$ = stars accrued in window; $\Delta\#$ = forks accrued in window). All five saturate the sig_score ceiling of 10.

Repository	Lang.	Score	$\Delta\star$	$\Delta\#$	Window	Description
Sunefei/PatchNet	Python	10	13	1	168 h	[TODO: Verify from GitHub repo page]
warpdotdev/Warp	—	10	211	26	168 h	AI-powered terminal emulator with collaborative and IDE-like features
Anil-matcha/Open-Higgsfield-AI	JavaScript	10	204	28	730 h	Open-source web interface for the Higgsfield AI cinematic video generation model
capcom6/android-sms-gateway	Kotlin	10	89	9	730 h	Self-hosted REST API gateway for sending and receiving SMS via an Android device
Fincept-Corporation/FinceptTerminal	Python	10	355	43	730 h	Terminal-based financial analytics platform for real-time market data and portfolio tracking

TimescaleDB for time-series storage, and a modern React frontend proved well-suited to the high-velocity, enrichment-heavy nature of the GitHub event stream. A central contribution is the Hidden Gem discovery engine, which applies a Poisson significance model to the continuous event stream to surface repositories, users, and organisations exhibiting statistically anomalous activity across daily, weekly, and monthly time windows. [TODO: Add 1–2 sentences with concrete findings from Section 5.5: e.g., cohort precision per window, total number of distinct hidden gems identified, and any notable patterns such as language distribution or temporal concentration.]

Several directions remain for future work:

- **Stream analytics.** Attaching Apache Flink or Spark Structured Streaming to the `github.events.raw` topic would enable stateful, sub-second anomaly detection as an alternative to the current batch-snapshot approach, without modifying the existing pipeline.
- **Long-term archival.** A Kafka Connect S3 sink writing Parquet files to object storage would enable cost-efficient historical analysis beyond the 90-day TimescaleDB retention window.
- **Alerting integration.** Configuring Grafana alerting rules on the Hidden Gem snapshot tables could proactively notify users when a repository’s significance score crosses a predefined threshold, removing the need for manual inspection of the ranking dashboard.

Acknowledgments

The authors thank the PM4 module supervisors at ZHAW School of Engineering for their guidance throughout the project. [TODO: Add names of supervisors or any other individuals to acknowledge.]

References

- [1] Apache Software Foundation. 2025. Apache Kafka Documentation. <https://kafka.apache.org/documentation/>. Accessed: March 2025.
- [2] GitHub, Inc. 2025. GitHub REST API Documentation. <https://docs.github.com/en/rest>. Accessed: March 2025.
- [3] Georgios Gousios. 2013. The GHTorrent dataset and tool suite. <https://www.ghorrent.org/>. Accessed: March 2025.
- [4] Ilya Grigorik. 2012. GH Archive — a project to record the public GitHub timeline, archive it, and make it easily accessible for further analysis. <https://www.gharchive.org/>. Accessed: March 2025.
- [5] Ian Hickson. 2015. Server-Sent Events — W3C Recommendation. <https://www.w3.org/TR/eventsource/>. W3C Recommendation, 3 February 2015.

- [6] Jay Kreps. 2014. Question: Why is a kappa architecture sometimes considered better than lambda architecture. <https://oreil.ly/kappa-architecture>. *O’Reilly Blog* (2014).
- [7] Filipe Lamarque et al. 2022. An analysis of GitHub ecosystem using GH Archive data. *Empirical Software Engineering* 27, 4 (2022), 1–34.
- [8] Nathan Marz and James Warren. 2015. Big Data: Principles and best practices of scalable real-time data systems. In *Manning Publications*.
- [9] Sebastián Ramírez. 2025. FastAPI Documentation. <https://fastapi.tiangolo.com/>. Accessed: March 2025.
- [10] Timescale, Inc. 2025. TimescaleDB Documentation. <https://docs.timescale.com/>. Accessed: March 2025.
- [11] Vercel, Inc. 2025. Next.js Documentation. <https://nextjs.org/docs>. Accessed: March 2025.

A Full System Architecture

B Project Timeline

Table 10: Actual project timeline and task allocation

Week	Phase	Deliverable	Owner
KW 03	Setup	Kafka pipeline, TimescaleDB schema, Docker Compose skeleton	[TODO: name(s)]
KW 04	Exploration	GitHub event stream collection, EDA, schema refinement	[TODO: name(s)]
KW 06	Pipeline	Producer, consumer, geo-enrichment, FastAPI endpoints	[TODO: name(s)]
KW 10	Dashboard	Next.js dashboard, Grafana provisioning, SSE live feed	[TODO: name(s)]
KW 13	Finalization	Testing, documentation, final report, presentation	All

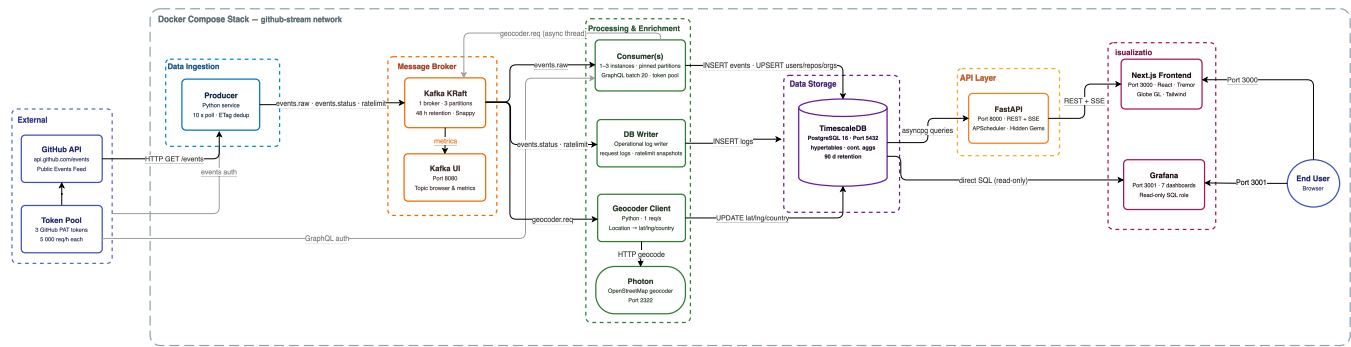


Figure 2: Full annotated system architecture diagram. All seven layers are shown with explicit data-flow labels, port numbers, and service boundaries of the Docker Compose stack.

C Database Schema

D Deployment Quick Reference

```
# 1. Clone and configure
git clone <repo-url> && cd pm4-github-insights
cp .env.example .env          # add GITHUB_TOKEN

# 2. Start all services
docker compose up --build

# 3. Access UIs
# http://localhost:3000 Next.js dashboard
```

```
# http://localhost:8000/docs FastAPI Swagger UI
# http://localhost:3001 Grafana (admin / admin)
# http://localhost:8080 Kafka-UI
```

```
# 4. Open a DB session
docker exec -it timescaledb \
psql -U github -d github_events

See docs/RUNBOOK.md and docs/ENV.md for the full operational
reference and environment variable documentation.
```

Received January 2025; revised [TODO: Fill in revision date]; accepted March 2025

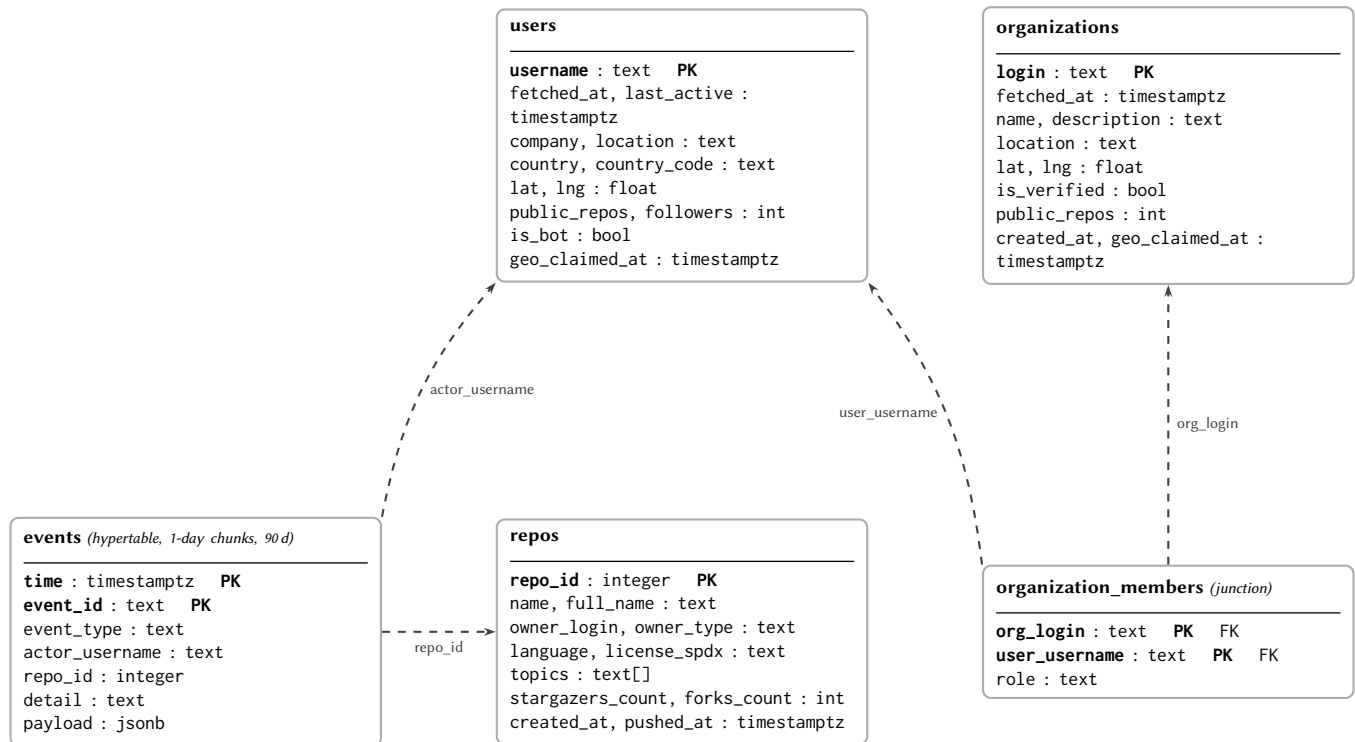


Figure 3: Core star schema. *events* is the central fact hypertable; *users*, *repos*, and *organizations* are dimension tables; *organization_members* is a junction table with enforced REFERENCES constraints. Dashed arrows from *events* to *users* and *repos* represent logical joins (no database-level FK constraint); arrows from *organization_members* represent actual REFERENCES constraints.

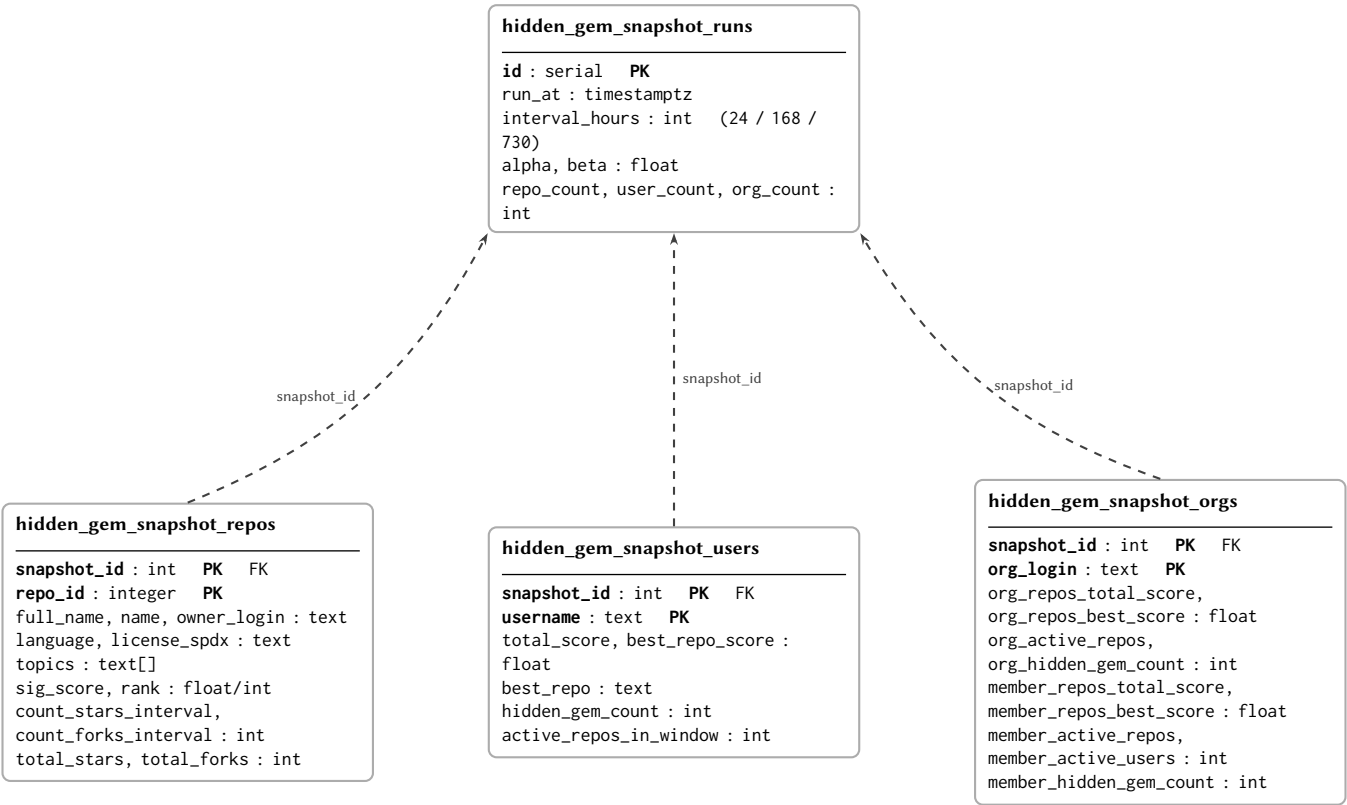


Figure 4: Hidden Gem snapshot tables. `hidden_gem_snapshot_runs` is the header record for each scoring run (one per interval per invocation); the three child tables store per-repo, per-user, and per-org significance scores for that run.

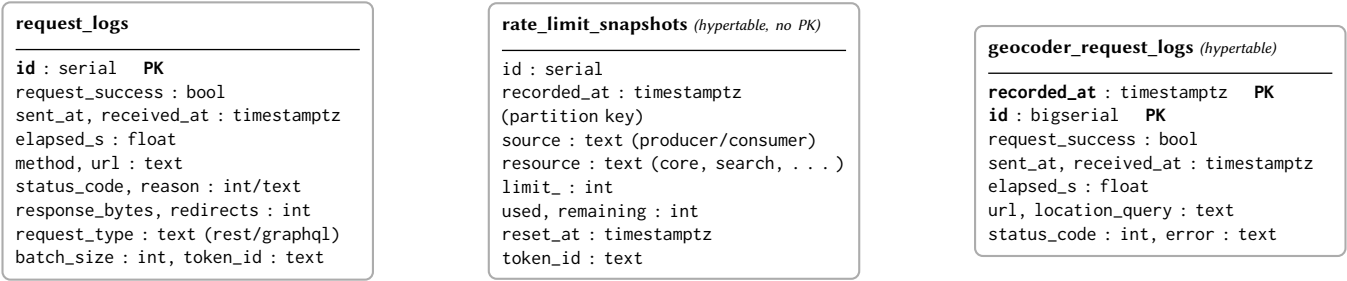


Figure 5: Operational log tables. These three tables record HTTP request logs for the GitHub REST/GraphQL API (`request_logs`), GitHub API rate-limit snapshots (`rate_limit_snapshots`), and Photon geocoder request logs (`geocoder_request_logs`). They carry no foreign key references to the core schema and are used exclusively for operational monitoring.